

MPC-PiNNs technique applied to Quadrotors

Authors

January 13, 2025

Todo list

crazyflie pic + brief description	2
uav simplified model	2
uav complete model	2
mpc structure	2
application of mpc to unicycle and quadrotor	3
definition of losses	5
image of pinn time	5
pipeline (prendi foto paper manipulator)	5
what we have obtained	5
problems	5
pinn vs pinnc	5

Contents

Todo list	1
1 Introduction	2
2 Model Predictive Control	2
3 Physical Informed Neural Network	4
4 Results	5
5 Conclusions	5

1 Introduction

crazyflie pic + brief description

Model Predictive Control (MPC) has emerged as a powerful control strategy in robotics due to its ability to handle constraints and optimize system behavior over a prediction horizon. When combined with neural networks, which can learn complex system dynamics, it creates a promising framework for controlling nonlinear systems.

This report presents the implementation and analysis of a Neural Model Predictive Control (NMPC) scheme applied to a quadrotor. The unicycle serves as an excellent test case due to its nonlinear dynamics, making it both challenging and representative of real-world robotic systems. Besides, we followed a bottom-up approach, testing the pipeline first on a unicycle, which still has a nonlinear dynamics, but is easier to treat due a simpler model and faster prototyping possibilities.

Our approach integrates Physical Informed Neural Network to approximate the quadrotor's dynamics, which are then utilized within an MPC framework to generate optimal control inputs. We explore the effectiveness of this method in trajectory tracking tasks, comparing it with traditional control approaches.

The following sections detail the theoretical foundations, implementation specifics, and experimental results of our neural MPC system. We also discuss the challenges encountered and potential improvements for future work.

2 Model Predictive Control

uav simplified model

uav complete model

mpc structure

Model Predictive Control (MPC), it is a control technique used to solve an optimization problem at each sampling instant. The controller predicts the future behavior of the system using a mathematical model and determines a sequence of control inputs that minimize a predefined cost function while satisfying state and input constraints.

$$\min_{u(0), u(1), \dots, u(N-1)} \sum_{k=0}^{N-1} \ell(x(k), u(k)) + \ell_f(x(N)) \quad (1)$$

$$\text{s.t. } x(k+1) = f(x(k), u(k)), \quad k = 0, \dots, N-1, \quad (2)$$

$$x(k) \in \mathcal{X}, \quad u(k) \in \mathcal{U}, \quad k = 0, \dots, N-1, \quad (3)$$

$$x(0) = x_0 \quad (4)$$

One of the main advantages of this control approach is the ability to handle multivariable interactions and constraints. However, solving the above optimization problem in real time can be computationally challenging, especially for large-scale or highly nonlinear systems. This is one reason why neural networks have been employed to either approximate the system model or enhance the optimization process. In our case, we have used Neural Networks to predict the next state after the application of the control input given by the MPC.

application of mpc to unicycle and quadrotor

$$\mathbf{R}_{\text{RPY}} = \begin{bmatrix} \cos \psi \cos \theta & \cos \psi \sin \theta \sin \phi - \sin \psi \cos \phi & \cos \psi \sin \theta \cos \phi + \sin \psi \sin \phi \\ \sin \psi \cos \theta & \sin \psi \sin \theta \sin \phi + \cos \psi \cos \phi & \sin \psi \sin \theta \cos \phi - \cos \psi \sin \phi \\ -\sin \theta & \cos \theta \sin \phi & \cos \theta \cos \phi \end{bmatrix} \quad (5)$$

$$\dot{\mathbf{v}} = \begin{bmatrix} 0.0 \\ 0.0 \\ -g \end{bmatrix} + \mathbf{R}_{\text{RPY}} \cdot \begin{bmatrix} 0.0 \\ 0.0 \\ \frac{T}{m} \end{bmatrix} \quad (6)$$

In our implementation, we have used a simplified dynamic model for the quadrotor instead of the simplified second order system.

$$\begin{cases} \ddot{x} = -(\cos \psi \sin \theta \cos \phi + \sin \psi \sin \phi) \frac{T}{m} \\ \ddot{y} = -(\sin \psi \sin \theta \cos \phi - \cos \psi \sin \phi) \frac{T}{m} \\ \ddot{z} = -(\cos \theta \cos \phi) \frac{T}{m} + g \\ \ddot{\phi} = \frac{\tau_{\phi}}{I_x} \\ \ddot{\theta} = \frac{\tau_{\theta}}{I_y} \\ \ddot{\psi} = \frac{\tau_{\psi}}{I_z} \end{cases} \quad (7)$$

$$\begin{cases} \dot{x} = v_x \\ \dot{y} = v_y \\ \dot{z} = v_z \\ \dot{v}_x = (\cos \psi \sin \theta \cos \phi + \sin \psi \sin \phi) \frac{T}{m} \\ \dot{v}_y = (\sin \psi \sin \theta \cos \phi - \cos \psi \sin \phi) \frac{T}{m} \\ \dot{v}_z = (\cos \theta \cos \phi) \frac{T}{m} - g \\ \dot{\phi} = \\ \dot{\theta} = \\ \dot{\psi} = \end{cases} \quad (8)$$

3 Physical Informed Neural Network

Traditional methods for solving differential equations, such as finite element or finite difference methods, often require discretization of the domain and can be computationally expensive for high-dimensional problems. Physical Informed Neural Networks (PiNNs) propose an alternative approach where the solution is approximated by a neural network that is trained not only on data but also by enforcing the underlying physical laws via the differential equations. The overall loss function $\mathcal{L}$ used is composed by the sum of two terms, which are the data loss $\mathcal{L}_{data}$ and the physics loss $\mathcal{L}_{phy}$.

$$\mathcal{L}(\theta) = \mathcal{L}_{data}(\theta) + \mathcal{L}_{phys}(\theta), \quad (9)$$

The first term defines the discrepancy between the prediction of the network and the observed data, while the second one specifies the correct modeling of the differential equation. Usually $\mathcal{L}_{data}$ is defined as the mean squared error between the prediction and the ground truth, while $\mathcal{L}_{phy}$ is defined using the residual of the differential equation.

$$\mathcal{L}_{data}(\theta) = \frac{1}{N_u} \sum_{i=1}^{N_u} |u_{\theta}(\mathbf{x}_i, t_i) - u_i^{obs}|^2 \quad (10)$$

$$\mathcal{L}_{phys}(\theta) = \frac{1}{N_r} \sum_{j=1}^{N_r} |\mathcal{N}[u_{\theta}](\mathbf{x}_j, t_j)|^2, \quad (11)$$

As previously stated, we have used a bottom-up approach by first testing this approach to a Unicycle. This robot is characterized by simple kinematic equations.

Its configuration vector is composed by the cartesian position (x, y) and its orientation angle θ , while the control inputs are the driving velocity v and the steering velocity w .

$$\begin{cases} \dot{x} = v \cos(\theta) \\ \dot{y} = v \sin(\theta) \\ \dot{\theta} = w \end{cases} \quad (12)$$

We have used the PiNN to predict the next state q_{k+1} of the robot given the current state, the control input given by the MPC and the time instant. Over a small time instant we have chosen to use Euler integration to do the approximation of the next state: $q_{k+1} = q_k + \Delta t \cdot f(q_k, u_k)$, where the dynamic function is defined as follows.

$$f(q_k, u_k) = \begin{cases} x_{k+1} = v_k \cos(\theta_k) \\ y_{k+1} = v_k \sin(\theta_k) \\ \theta_{k+1} = w_k \end{cases} \quad (13)$$

definition of losses

image of pinn time

pipeline (prendi foto paper manipulator)

4 Results

what we have obtained

problems

pinn vs pinnc

5 Conclusions